

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #3
CS@TUCN

Computer Science

Agenda

- **Example – length of a list**
 - Introducing recursion types
 - Types of parameters
- **Example – reversing a list**
 - Introducing pattern position
 - Meaning of arguments
- **Backtracking prevention – the CUT (!)**
 - Negation (as failure)

Example 4 – how to recurse?

- Determine the length of a list, number of arguments?

length1/2 (List, Length)

```
length1([],0).
```

% length of empty list is 0

```
length1([_|T],N):-
```

```
    length1(T,N1),
```

% length of tail calculated as N1

```
    N is N1+1.
```

% add 1 to the length of tail

- Order of clauses? This way? Swapped? Which is correct? Which is better? Why? Is it important? Why?
 - (discuss indexation on the first argument. Oral explanation.)
- Meaning of N1?
 - partial result (N1) not quite relevant (is length of the tail)
- What if we need to keep the number of “consumed” elements?
- This solution uses **backward** recursion, meaning that the **result is built as recursion RETURNS**

List length – forward recursion

- What if we build it while advancing?

```
length2/2 (Length, List)
```

```
length2 (PR, [_|T]) :-
```

```
    NPR is PR+1,
```

% add 1 to the partial result so far

% i.e. length of the covered part

```
    length2 (NPR, T) .
```

% go count the rest of the elements

```
length2 (R, []) .
```

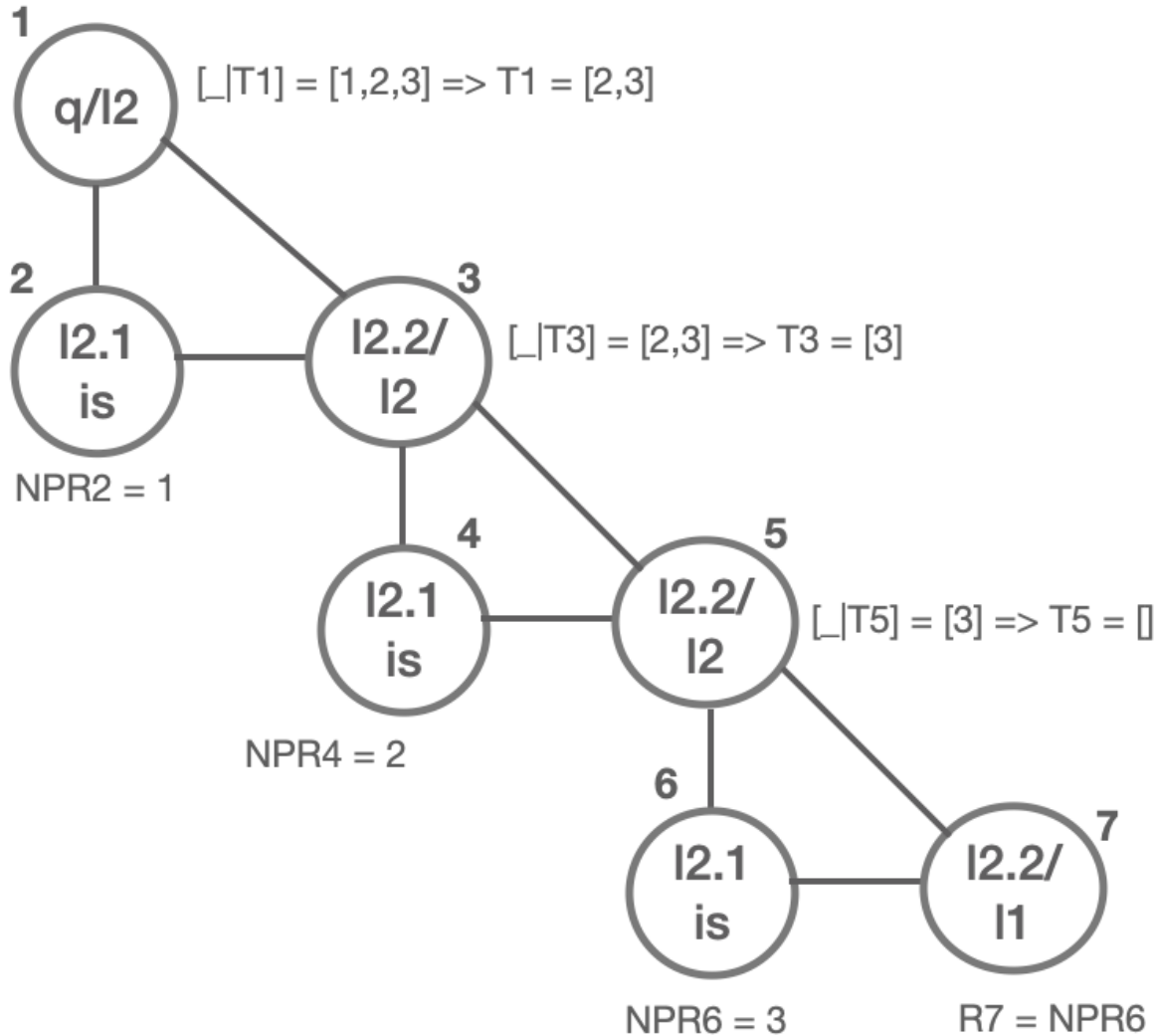
% R has the final list length

- What's the difference compared to the previous solution?
- PR represents here what we *have covered*; before – what we've *yet to cover*.

length2/2

Deduction tree

```
[11] length2([],R) .
[12] length2([_|T],PR):-
[121]   NPR is PR+1,
[122]   length2(T, NPR) .
[q] ?- length2([1,2,3], 0) .
```



List length – forward recursion

```
length2/2 (list, length)
```

- Not enough – as result, although calculated, is NEVER reported

```
length2([],N) .
```

```
length2([_|T],N) :-
```

```
    N is N1+1,
```

```
    length2(T,N1) .
```

- Upgrade it with 3rd argument = “steal” the result from the rightmost leaf

```
length2([],PR,FR) :-FR=PR.
```

```
length2([_|T],PR,FR) :-
```

```
    NPR is PR+1,
```

```
    length2(T,NPR,FR) .
```

- Observe that in the second clause does not modify FR at all

List length – forward recursion

- With default unification:

```
length2([], Res, Res) .  
length2([_ | T], PR, FR) :-  
    NPR is PR+1,  
    length2(T, NPR, FR) .
```

- How should we call it? Needs specific INITIALIZATION:

```
?- length2(InList, 0, Result) .
```

- To avoid mandatory initial call, use a wrapper:

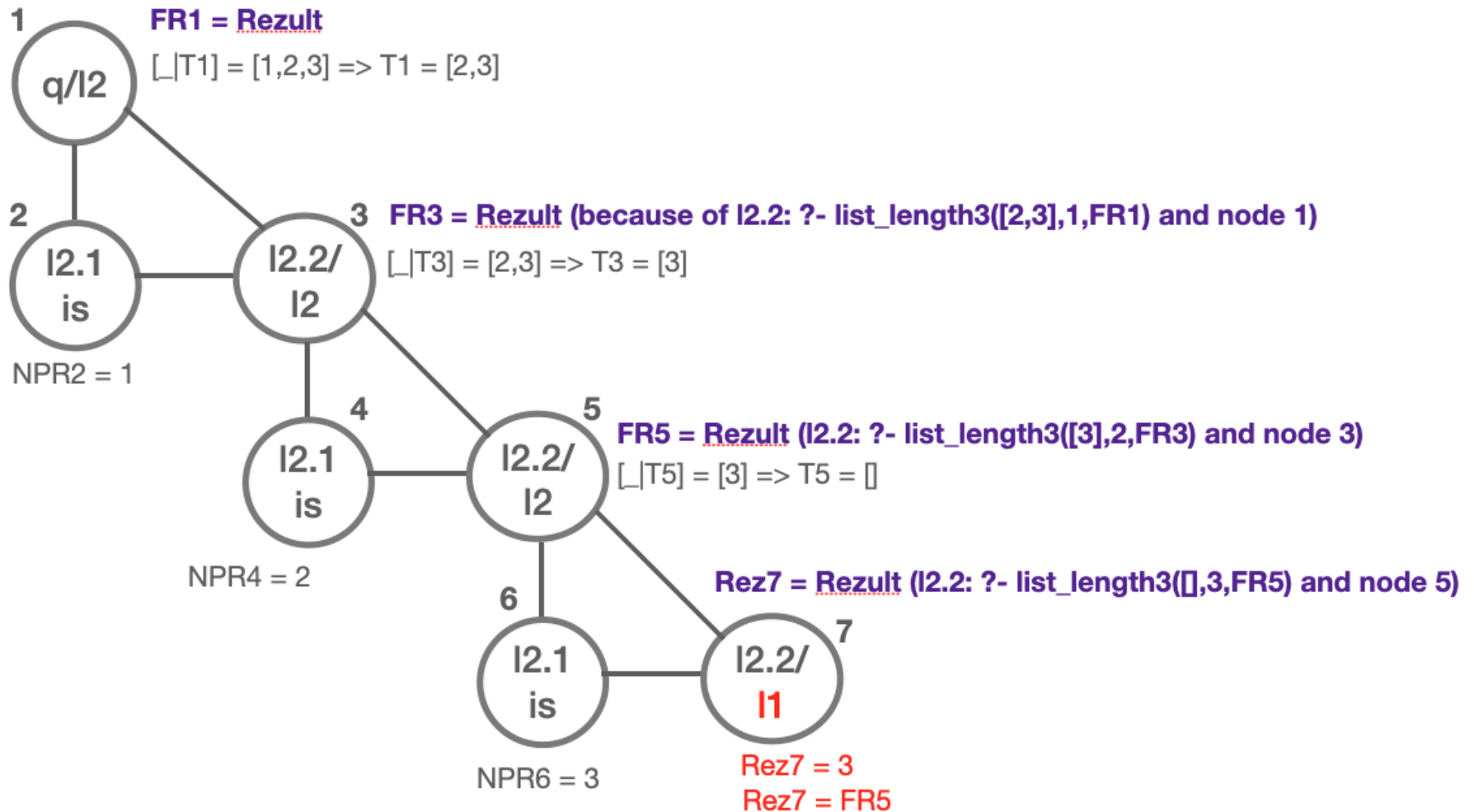
```
length2(InList, Result) :-  
    length2(InList, 0, Result) .
```

- Can have same name? why/why not?
- Discuss signature definition
- Thus, the query becomes: ?- length2(InList, Result) .

Length2/3 deduction tree

```
[11] length2([],Res,Res).
[12] length2([_|T],PR,FR):-
[121]     NPR is PR+1,
[122]     length2(T,NPR,FR).

[q] ?- length2(InList,0,Result).
```



Forward vs Backward recursion

Forward recursion

+

- Potentially useful partial results
- Efficiency if parallelized
- LCO (Last Call Optimization)

-

- Need for extra variable (final result)
- Need for specialized call (for accumulator initialization)

Backward recursion

- No need for extra variable
- No need for special call

- Rarely useful intermediate results
- Not efficient if parallelized (execution of concurrent process(es) - delayed until recursion returns)

Computer Science

Reversing a list

- How many arguments?

- 2 = list + reversed list

```
reverse1/2 (List, Rev_list)
```

```
reverse1([], []). % empty input returns empty output
reverse1([H|T], R) :-
    reverse1(T, PR), % reverses the tail of list
    append(PR, [H], R). % concatenates the partial results
```

- Needs specialized call? Why/why not?
- What type of recursion is here?
 - *Backward*
- Efficiency? How is it calculated?
- Can we avoid 2 traversals through the list? How?

Reversing a list – forward recursion

- How many arguments?

```
reverse2/3(List, Part_rev_list, Rev_list)
```

```
reverse2([], PR, PR) .
```

% when input gets empty the partial
% result becomes the final one

```
reverse2([H|T], PR, R) :-  
    NPR=[H|PR],
```

% concatenates the partial results
% PR, NPR acts as a stack

```
reverse2(T, NPR, R) . % reverses the tail of list
```

- Wrapper (MUST have)

```
reverse(In, Out) :-
```

```
reverse2(In, [], Out) .
```

- Efficiency? Is it better?

Reversing a list – forward recursion

```
reverse2([], PR, PR) .  
reverse2([H|T], PR, R) :-  
    reverse2(T, [H|PR], R) .
```

- Same as before, but explicit → implicit unification
- Let's change the order of arguments 2 and 3 (makes no difference in the functionality)

```
reverse2([], PR, PR) .  
reverse2([H|T], R, PR) :-  
    reverse2(T, R, [H|PR]) .
```

- Rename variables:

```
reverse2([], L, L) .  
reverse2([H|T], L, R) :-  
    reverse2(T, L, [H|R]) .
```

Comparative analysis

```
reverse([], L, L).
reverse([H|T], L, R) :-
    reverse(T, L, [H|R]).

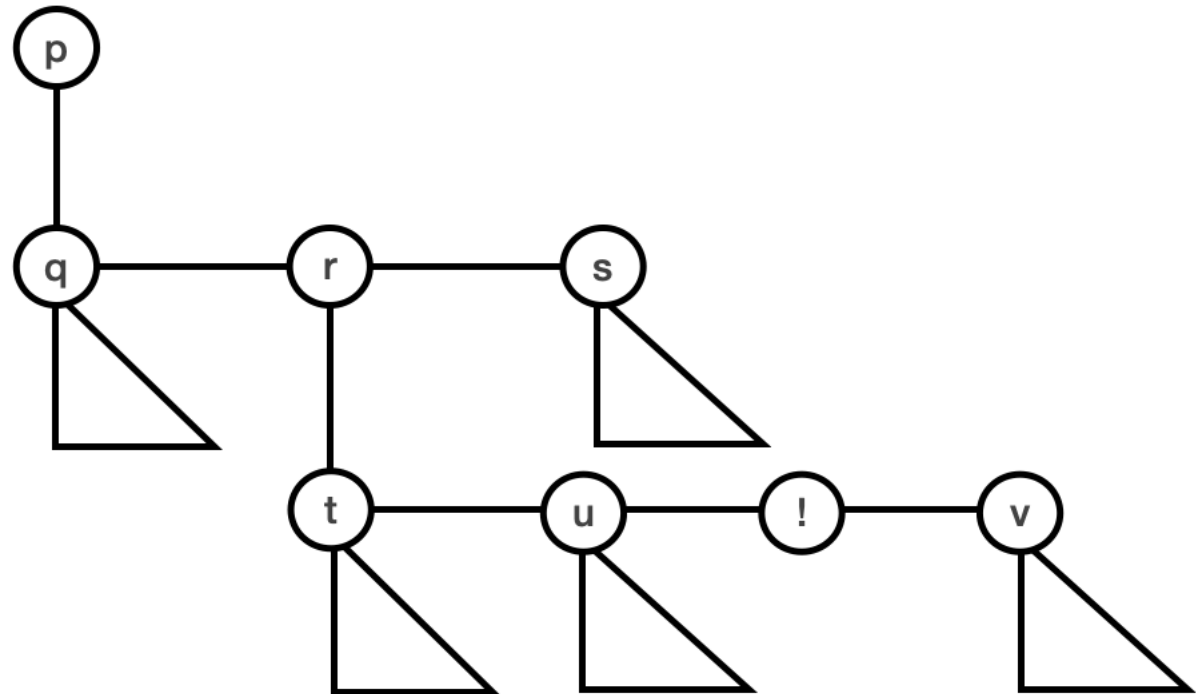
append([], L, L).
append([H|T], L, [H|R]) :-
    append(T, L, R).
```

- What is the difference?
- Meaning of pattern's position
- Length of arguments analysis

Backtracking avoidance (the CUT !)

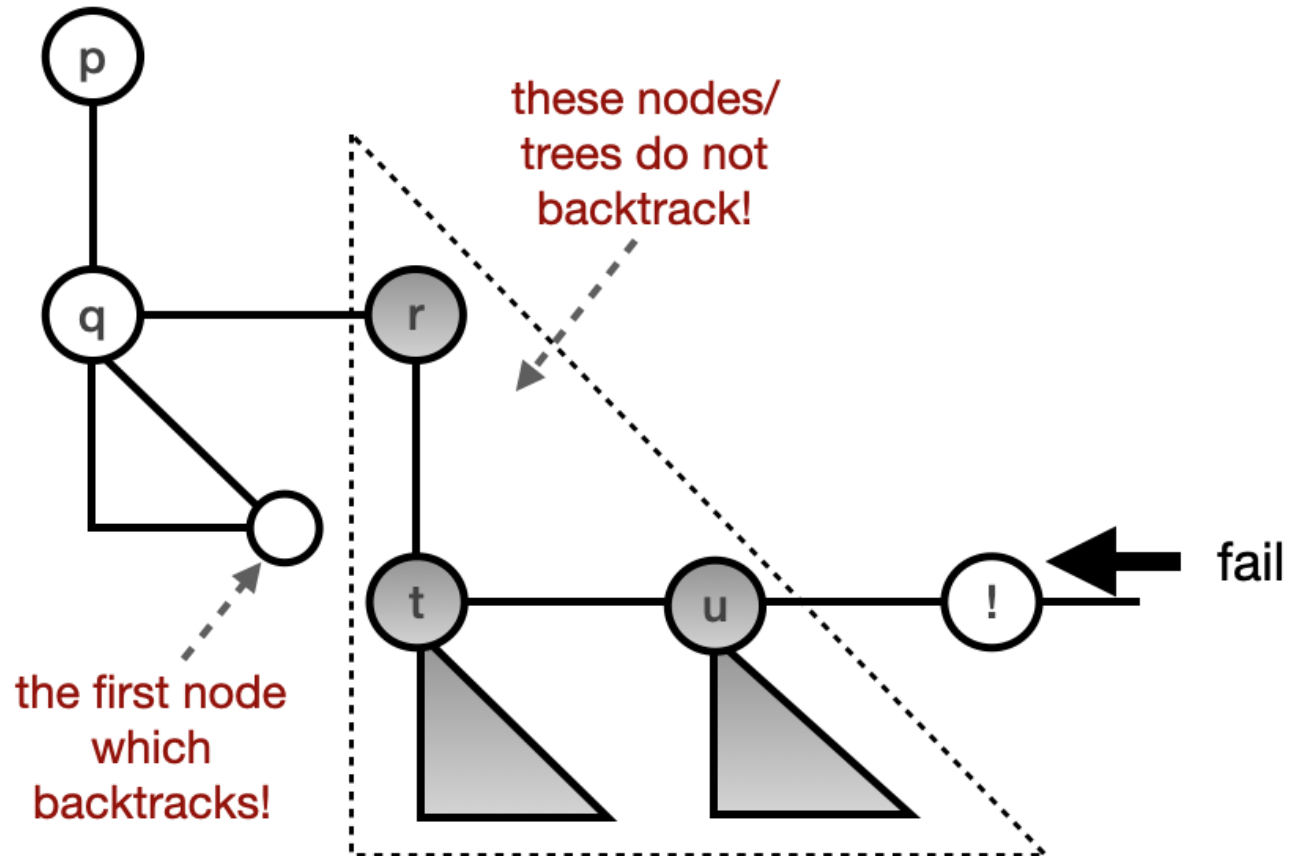
- Built-in predicate
- Always succeeds
- Never backtracks

$p:-q, r, s.$
 $r:-t, u, !, v.$



The CUT (!)

$p:-q,r,s.$
 $r:-t,u,! ,v.$



The CUT (!) – effects

- **!** Prevents the backtracking of the following categories:
 - All left siblings
 - All subtrees of left sibling nodes
 - Parent node
- NO other node is affected:
 - Siblings on the right/their subtrees have the ability to backtrack
 - Left siblings DO backtrack BEFORE **!** Is executed
 - Their subtrees also
- As everything, **!** takes effect ONLY AFTER its execution (before execution it does NOT actually exist)

Negation (as failure)

- Unfair behavior
- As it assumes the **inability to prove True means False**
- Isn't it so? Why? Postponed discussion.

`not (P) :-`

`P, !, fail.`

`not (P) .`

- $P - \text{true} \Rightarrow \text{not}(P) - \text{false}; \text{OK.}$
- $P - \text{false} \Rightarrow \text{not}(P) - \text{true}; \text{OK.}$
- $P - ?$ Don't know how it is. Why? Incomplete information, not known at the time.

How is `not(P)`? Let's run it:

P's execution fails (P is NOT yet True),
we enter the second clause and succeeds.

Negation (as failure)

```
not (P) :-  
    P, !, fail.  
not (P) .
```

- Negation - assumes negated is True just because we don't have information about P.
- Negation as failure (to check validity)